

AI, Agents & AI Coding Tools: A Practical Deep Dive

How AI agents, multi-agent orchestration, and modern coding tools are changing software development. A hands-on look for engineering and IT teams.

SPEAKER

Sebastian Kehle

ROLE

AI Engineer & Founder, webvise

DURATION

60 Minutes

FORMAT

Talk + Q&A

Sebastian Kehle

Founder of webvise and AI Engineer at luca in Berlin. My background is in design and communication - over the years I've moved deeper into software engineering and automation.

At luca, I work on LLM-based solutions for the hospitality industry. At webvise, I combine that experience with client projects: web development, design, and increasingly AI automation.

What I'm presenting today are tools and workflows I use in my daily work.

TypeScript

React

Next.js

Node.js

PostgreSQL

OpenAI

Anthropic

Vercel AI SDK

Mastra

n8n

2025 - present	AI Engineer, luca LLM integrations, intelligent tooling, automation
2022 - 2025	Frontend Engineer, luca B2B features, design collaboration
2016 - present	Founder, webvise Design, development, AI automation
2017 - 2021	CEO, Addisco Education Recruiting platform, matching algorithm

Why This Matters Right Now

The Inflection Point

We've crossed from AI as novelty to AI as infrastructure. Every major IDE now ships with an AI assistant built in. Terminal agents work autonomously on complex, multi-file tasks. The question has shifted from "should we use AI?" to "how fast can we adopt it responsibly?"

Teams that move now build compounding advantages. Teams that wait hand competitors a head start that grows every quarter.

The productivity gap between AI-assisted and traditional development is widening faster than any previous technology shift in software engineering.

20-45%

Productivity gain in code generation
McKinsey, 2025

55%

Faster task completion with AI
GitHub, 2025

76%

of developers already use AI tools
Stack Overflow, 2025

75%

of enterprise engineers with AI tools by 2028
Gartner, 2026

Three Waves of AI-Assisted Development

From autocomplete to autonomous agents. The pace of change outstrips every previous technology adoption cycle in software.

Wave 01 Autocomplete 2021-2023. GitHub Copilot brings AI into the editor. Line completion, boilerplate generation. Helpful but limited to single lines and functions. The developer stays fully in control at every step.	Wave 02 Chat & Copilot 2023-2025. ChatGPT, Claude, and specialized assistants. Conversations about code, multi-file context. Cursor and Windsurf integrate AI deeply into the IDE - not just a sidebar, but a real collaborator.	Wave 03 Autonomous Agents 2025-present. AI systems that independently plan, use tools, test, and deploy. Not one prompt, one answer - but one goal, many steps executed without hand-holding at each stage.
---	--	---

Most teams are still in Wave 1 or 2. Wave 3 is available today and the productivity delta is significant. Each wave builds on the previous - none of them obsolete what came before.

The AI Coding Tool Landscape

Co GitHub Copilot Broadest adoption, solid autocomplete quality. Agent Mode since 2025 but feels bolted on rather than native. Good for simple completion - hits limits quickly on complex, multi-step tasks.	Cu Cursor VS Code fork with native AI integration. Strong multi-file editing, good codebase context, Composer for complex tasks. Currently one of the best overall IDE experiences available.	Cl Claude Code Terminal-based autonomous agent. Plans, implements, and tests independently. Excellent codebase understanding. Full Git, shell, and API integration. API-based and self-hostable.
Wi Windsurf AI-first IDE with a Flows system for multi-step tasks. Low barrier to entry, good context management. Solid alternative to Cursor, especially for teams new to AI-assisted development.	Cx Codex CLI OpenAI's terminal agent. Cloud sandbox execution, good integration with the OpenAI ecosystem. Still relatively young compared to the others but developing quickly.	Ge Gemini CLI Google's terminal agent. Largest context window (1M+ tokens), deep Google ecosystem integration. Free tier available, making it accessible for experimentation and initial evaluation.

How They Compare

Visual comparison across five criteria. Ratings reflect real-world usage patterns as of Q1 2026, not marketing claims.

CRITERION	GITHUB COPILOT	CURSORM	CLAUDE CODE	WINDSURF
Autonomy				
Codebase Understanding				
Complex Tasks				
Enterprise Features				
Compliance Controls				

The landscape evolves rapidly. Verify with current documentation before procurement decisions.

What Is an AI Agent?

A chatbot answers questions. An agent completes tasks. That is a fundamental difference - not a matter of degree.

A chatbot waits for your next message. An agent receives a goal, decomposes it into steps, executes those steps using tools, checks the results, and iterates until done - without waiting for you to prompt each step.



Goal Understanding

Understands complex goals and independently breaks them down into executable steps.



Tool Usage

Filesystem, APIs, databases, browsers, terminal - all accessible and combinable.



Iteration

Checks results, corrects course, tries alternatives until the objective is met.

The Agent Loop

01 Goal Receives and interprets the objective

02 Plan Decomposes the goal into executable steps

03 Execute Uses tools to carry out each step

04 Verify Checks output against the original goal

05 Iterate Adjusts and repeats until complete

Model Context Protocol - USB-C for AI

MCP is an open standard that defines how AI models communicate with external tools and data sources. One protocol, all tools - no more bespoke integrations for every tool-model combination.

Anthropic published the spec, and the ecosystem has adopted it rapidly. Claude Code, Cursor, Windsurf, and others all support MCP. Your server works across all of them.

Before MCP	With MCP
• Every tool required its own custom integration • No standardization across agents • No reusability between tools • Fragmented, proprietary ecosystem	• One protocol for all tools • Interchangeable and composable • Build once, use everywhere • Open, growing ecosystem

Real MCP Server Examples

DB	Database Server PostgreSQL, Redis - query data, run migrations, inspect schemas directly from the agent
GIT	Git Server Repos, branches, PRs, commit history - full version control access beyond the local repo
PM	Jira / Linear Server Read tickets, update status, create issues from code context automatically
MON	Monitoring Server Grafana, logs, alerts - bring observability into the agent's context window
	Browser Server Navigation, screenshots, testing - automate the full web stack end to end

My Daily Setup: Claude Code + OMC

Claude Code

Claude Code runs in the terminal with access to the entire project filesystem, Git, shell commands, and configured MCP servers. It's my primary development tool - not a plugin or IDE extension, but a full autonomous agent that lives in the terminal alongside the rest of my workflow.

It understands the entire codebase, not just the file you have open. It can run tests, commit code, open PRs, and check deployment status without leaving the terminal.

Access	Capabilities
• Full project filesystem • Git history and branches • Shell and CLI tools • Configured MCP servers	• Read and write any file • Run tests and builds • Create commits and PRs • Query internal systems

oh-my-claudecode (OMC)

OMC adds a multi-agent orchestration layer on top of Claude Code. Instead of one agent doing everything sequentially, specialized agents coordinate: a Planner maps the work, Executors implement in parallel, a Reviewer checks quality, a Verifier confirms everything holds.

Skills - reusable Markdown instructions - give every agent the same institutional knowledge: our stack conventions, QA standards, and delivery process.

This is not a demo setup. This is how I ship production code every day.
The entire webvise site is built and maintained this way.

Specialized Agents

Each agent has a focused role and clear constraints. Specialization beats generalization when tasks have well-defined boundaries.

 Architect Strategic architecture decisions. Read-only access - reviews plans before any code is written. Prevents expensive structural mistakes early, when they're cheapest to fix.	 Executor Focused implementation. Works in parallel on independent tasks. Constrained to the smallest viable change - no scope creep, no "while I'm here" additions.	 Code Reviewer Detailed reviews with severity ratings. Finds patterns humans miss when tired or under deadline pressure. Applies consistent standards on every single review.
 Test Engineer Test strategy and TDD. Writes and validates tests automatically. Treats failing tests as signals about implementation quality, not obstacles to route around.	 Security Reviewer OWASP Top 10, secrets detection, unsafe patterns. Runs on every significant change - not just before release, when fixing issues is most expensive.	 Verifier Evidence-based completion checks. Tests passed, no regressions, deployment healthy. Never declares done without proof - no assumptions accepted.

The key insight: agents that specialize and have clear constraints produce better results than one generalist agent trying to do everything at once.

From Requirement to Deployment

A concrete example showing how agents coordinate on a realistic feature request.

```
// Requirement: "Add user authentication to the API" 1. Planner → Analyzes existing architecture, identifies auth patterns, maps all dependencies
2. Architect → Reviews plan, recommends JWT + session strategy, flags security considerations 3. Executors → Work in parallel on independent
pieces: | Agent 1: Auth middleware + route guards | Agent 2: User model + database migration | Agent 3: Test suite + API documentation 4.
Reviewer → Checks code quality, security implications, consistency with codebase patterns 5. Verifier → All tests green, no regressions, auth
flow works end-to-end, ready to merge
```

Total time: ~45 minutes for what typically takes 1-2 days. The parallel execution and built-in review loops are the multiplier.

The developer's role shifts: from writing code to reviewing agent output, providing domain context, and making architecture decisions. Higher leverage, same accountability.

Skills: Reusable Agent Capabilities

Skills are Markdown-based instructions that give agents specific capabilities. They encode institutional knowledge - your conventions, your standards, your processes - in a format any agent can consume on any supported tool.

Works in: Claude Code, Cursor, Copilot, Windsurf, and 19+ other AI tools.

No vendor lock-in. Version-controlled in Git like any other piece of code.

skills.sh Marketplace

An open marketplace for AI agent skills. Teams create, share, and discover skills - instead of every team writing their own prompts from scratch.

For enterprises: define a "Security Review Skill" once, and every developer automatically applies the same standard across all their tools.

- Portable
- Versionable
- Composable
- Team-shareable

Examples from Practice

webvise-conventions Tech stack conventions, design system rules, code patterns. Every agent knows our standards without being told twice.	webvise-qa 7-category QA audit covering Performance, Accessibility, SEO, Responsive, Analytics, Security, and UX.
webvise-deliver 6-phase delivery workflow: Discover, Plan, Build, SEO, QA, Deliver. Consistent process every engagement.	Enterprise Skills Coding standards, review checklists, onboarding guides. In Git, versioned, available to every team member.

Case Study: AI-Assisted Client Delivery

The Challenge

A client needed a complete web platform with an integrated AI chatbot, multilingual support across 8 languages, and automated content workflows. Traditional project estimate for a small team: 6-8 weeks.

The platform needed to be performant, SEO-optimized from day one, and maintainable long-term without a large engineering team behind it.

TRADITIONAL ESTIMATE

6-8 weeks

2-3 developers, sequential delivery

With AI-Assisted Development

Next.js platform with Gemini-powered AI chatbot, 8-language support via automated pipelines, performance score 95+ from launch. Test coverage included from day one, not added later.

- Multi-agent parallel implementation throughout
- Automated QA agent ran on every change
- Skills encoded all conventions once, agents inherited them
- Verifier confirmed quality before every client handoff

ACTUAL DELIVERY

2 weeks

1 developer, parallel AI agents

Next.js

Gemini API

i18n

Claude Code

OMC

Honest Limitations

Credibility requires honesty. Here is a clear-eyed view of where AI genuinely helps and where human judgment remains essential.

AI excels at

- Code generation from clear specifications
- Writing comprehensive, consistent test suites
- Documentation that stays current with the code
- Refactoring and modernizing legacy code
- Pattern recognition across large codebases
- Dependency management and security patches
- Code review - thorough, consistent, tireless
- Repetitive implementation and boilerplate

AI still needs humans for

- Architecture decisions with real business impact
- Business logic requiring domain expertise
- User research and understanding real needs
- Product strategy and prioritization
- Edge case judgment in ambiguous situations
- Team dynamics and stakeholder management
- Ethical considerations and value tradeoffs
- Novel problems without existing precedent

The best mental model: treat AI as a highly capable junior developer - fast, thorough, and tireless, but needing guidance, review, and someone who understands the bigger picture.

EU AI Act: What You Need to Know

Full application from August 2026. AI systems are classified by risk level - and the classification of your development tooling matters.



Minimal Risk

Most coding tools fall here. Autocomplete, refactoring, documentation generation.

Transparency obligations apply - users should know AI is involved in producing output.



Limited Risk

Agent systems that autonomously deploy or make infrastructure changes. Documentation requirements and human oversight mechanisms are required at this tier.



High Risk

Safety-critical software, systems affecting fundamental rights. Mandatory human supervision, complete audit trails, and conformity assessment before deployment.

Three immediate actions: (1) Classify your applications and tools by risk level now. (2) Define policies per risk tier - what is allowed, what requires human review. (3) Establish an AI governance board before August 2026.

The Act doesn't ban AI in development - it establishes accountability. Most development tool usage falls in minimal or limited risk. Get ahead of it now rather than scrambling in 2026.

Key question: does your AI tool make decisions, or assist humans in making decisions? The answer changes your risk classification significantly.

GDPR, Data Flows & Provider Landscape

Where Does Your Code Go?

When you use a cloud AI tool, your code - including business logic, proprietary algorithms, and credentials if you are careless - leaves your network. Understanding where it goes is essential for GDPR compliance and IP protection.

PROVIDER	TRAINING	DPA AVAILABLE	EU HOSTING
Anthropic	No (API)	Yes	Planned
OpenAI	No (API)	Yes	Azure EU
GitHub	No (Ent.)	Yes	No
Google	No (API)	Yes	Yes

Practical Recommendations

- Always use API access - API agreements typically offer stronger data protections than consumer products
- Sign DPAs with every provider before any production use
- Scan before sending - use pre-commit hooks to detect secrets and sensitive patterns
- Evaluate EU hosting for regulated industries - options are expanding rapidly in 2025-2026
- Classify data sensitivity - not all code carries the same risk level

For highly regulated environments, local models are available and increasingly competitive. See the following slide for details.

The Air-Gap Option: Local AI Models

Why Local Models Matter

Full on-premise deployment. Zero data leaves your network. No DPA negotiations, no EU hosting questions, no third-party dependencies for sensitive work. The model runs on your hardware, processes your code, and results stay internal.

Best suited for: safety-critical codebases, IP-sensitive projects, regulated industries such as banking, healthcare, and defense, and organizations with existing air-gap requirements.

Runtime Tooling

Ollama Simplest local model runner. Best developer experience for getting started in under 10 minutes.	llama.cpp Maximum performance on commodity hardware. CPU and GPU inference, highly portable.	vLLM Production-grade serving. High throughput for team and organization-level deployments.
--	--	---

Current Best Models for Coding

MODEL	SIZE	QUALITY
Qwen 2.5 Coder	7B - 32B	Very Good
DeepSeek V3	16B - 236B	Excellent
Codestral	22B	Very Good
StarCoder2	3B - 15B	Good

Hybrid approach: Local models for sensitive code and IP-critical work. Cloud APIs for general development where sensitivity is lower. A written policy governing which tier applies to what.

Local model quality is closing the gap with cloud APIs fast. DeepSeek V3 at full size is competitive with frontier cloud models for code tasks.

Audit Trails & Traceability

AI-assisted development must be as traceable as traditional development. With the right practices, it can be more so.



Git Integration

Normal workflows apply. Feature branches, pull requests, code review. Every AI-generated change goes through the same process as any human-written code.



Co-Author Tags

AI-assisted commits are marked with Co-Authored-By tags. Full, permanent transparency in Git history about which commits involved AI assistance.



Review Mandate

No AI-generated code merges without human review. The same standard as any contributor - no exceptions for small changes or quick fixes.

Logging Requirements

- Which model generated or assisted the code
- Prompt context (sanitized of secrets)
- Output before and after human review
- Who approved, when, and what changed

Quality Standards

- AI-generated code meets the same test coverage bar
- Same linting, type checking, and security scanning
- Performance budgets apply equally to all code
- Documentation required, not optional

A well-implemented AI development process produces better audit trails than purely human development - agents log their reasoning, not just their output.

Adoption: Crawl - Walk - Run

Structured rollout of AI coding tools. Step by step, measurable, controlled. The goal is sustainable adoption, not a failed big-bang rollout.

PHASE 01 / CRAWL Build Experience Months 1-3. 5-10 developers. • Start with Cursor or Windsurf (low barrier) • Code completion, docs, unit tests only • Define baseline policies and guidelines • Measure developer satisfaction and output quality • No autonomous agents yet	PHASE 02 / WALK Establish Standards Months 4-9. 50-100 developers. • Introduce Claude Code for complex tasks • First MCP servers for internal systems • Create a shared skills repository • Sign DPAs, establish AI policy in writing • Proxy layer for logging and filtering	PHASE 03 / RUN AI as Standard Month 10+. All teams. • Multi-agent workflows in production • Automated QA and review pipelines • Governance framework and regular audits • Measure ROI per team systematically • Champions program for ongoing adoption
---	--	---

"AI replaces developers" - this fear kills adoption before it starts. Reframe it: AI makes developers more effective. Champions program: identify AI-curious developers in each team and make them multipliers for the rest.

Start Here: Quick Wins

High-value, low-risk starting points that deliver measurable results without requiring full organizational commitment upfront.

Immediate (Week 1-4)

- **Legacy documentation.** Point an agent at undocumented code and generate READMEs, inline comments, and architecture notes. No risk, high value.
- **Test coverage.** Most codebases have coverage gaps. AI writes thorough test suites faster than humans. Start with the most critical paths first.
- **Code reviews.** Add AI review as a first pass before human review. It catches obvious issues so humans can focus on what actually requires judgment.
- **Dependency updates.** Automate security patches and version bumps. AI handles the mechanical work; humans approve significant changes.

Medium-term (Month 2-6)

- **MCP servers for internal systems.** Connect your ticket tracker, database, and monitoring to your agents. Dramatically expands what agents can complete autonomously.
- **Shared skills repository.** One canonical set of conventions every developer's agent inherits. Eliminates "which standard applies?" permanently.
- **AI-assisted incident response.** Agents that can query logs, identify patterns, and propose fixes during incidents - when speed matters most.
- **Legacy modernization.** Incremental, agent-assisted migration of old systems. High effort traditionally, much more tractable with AI.

Pick one quick win, measure the result clearly, then use that evidence to build the case for the next step. Evidence beats argument every time.

What Comes Next?

Short-term (2026)

- AI agents standard in every major IDE
- MCP as the de facto integration standard
- Local models reaching cloud-level quality
- Enterprise features and compliance controls mature across all providers

Mid-term (2027)

- Multi-agent teams as normal practice
- AI-assisted legacy migration at scale
- Specialized industry-specific models
- Compliance checks embedded directly in AI workflows

"The question isn't whether to adopt AI in development. It's how fast you can do it responsibly."

Teams that move thoughtfully now build a compounding advantage that grows every quarter.

5 Recommendations

1. **Start now.** The learning curve is shallower than you expect.
2. **Start small.** One pilot team, one tool, clear metrics.
3. **Invest in governance.** Define policies before you need them.
4. **Build knowledge, not just licenses.** Understanding beats access.
5. **Measure outcomes.** What isn't measured doesn't improve.

Let's Talk

Sebastian Kehle

I help companies integrate AI meaningfully into development processes
- from initial tool evaluation through to full multi-agent workflows in
production.

sebastian.kehle@webvise.io

webvise.io / sebastiankehle.com

LinkedIn: [/in/sebastiankehle](https://in.linkedin.com/in/sebastiankehle)

GitHub: [/sebastiankehle](https://github.com/sebastiankehle)

AI Strategy

Multi-Agent Workflows

MCP Integration

Team Enablement

FURTHER READING / WEBVISE BLOG

- [Best Local AI Models for Compliant Businesses](#)
- [How AI Agents Are Transforming Business Automation](#)
- [What Is the Model Context Protocol \(MCP\)](#)
- [AI Regulations and Certifications in Germany](#)
- [AI Agent Skills Directory](#)

OFFICIAL DOCUMENTATION

- docs.anthropic.com/claude-code
- modelcontextprotocol.io
- skills.sh
- digital-strategy.ec.europa.eu/en/policies/regulatory-framework-ai